

The Blessings of Overparameterization: Applications in Solving Economic Models

Mahdi E Kahou¹ Jesús Fernández-Villaverde²

¹Bowdoin College

²University of Pennsylvania

Deep Learning and Macroeconomics Conference (DLMacro 2026)
UNC Chapel Hill September 3–4, 2026

Motivation, Question, and Contribution

“I remember my friend Johnny von Neumann used to say, with four parameters I can fit an elephant, and with five I can make him wiggle his trunk.” – *Enrico Fermi*

- Deep learning has proved remarkably capable at solving **high-dimensional problems**.
- Economists are now using it to find accurate approximate solutions to dynamic models once thought intractable.
- These neural networks are **massively overparameterized**, with far more parameters than “data” points.
- The worry: more parameters than “data” points leads to **“overfitting”** and poor out-of-grid / out-of-distribution performance.

Question

When solving dynamic models with neural networks:

- Does overparameterization lead to poor approximation in out-of-grid / out-of-distribution performance?
- Is overparameterization a “necessary evil” that comes with modern machine learning methods, or does it give us something extra?
- One key difference from statistics: our residuals are deterministic evaluations of the functional equation, so there is **no noise to overfit**. The question is what the algorithm does **between the grid points**, and that is where we look.

1. **Accuracy improves with size:** as the number of network parameters grows, the solutions become more accurate.
2. **Algorithmic stability:** the approximate solutions of dynamic models in economics are unstable; overparameterization increases stability.
3. **A mechanism, in one dissectable case:** in a random-features version of the LQ model we can show exactly why: width always buys agreement; the implicit bias decides what you agree on.
 - and in the full neural networks, **width also improves trainability:** wider nets reliably reach the low-loss solution (the random feature isolates the rest by removing the optimizer).

Background: Economic Models, Approximate Solutions, and Overparameterization

An economic model is a system of functional equations

Many theoretical models can be written as functional equations:

- Economic object of interest: f , where $f : \mathcal{X} \rightarrow \mathcal{R} \subseteq \mathbb{R}^d$
 - e.g., entry/exit decision, investment choice, best-response, etc.
- Domain of f : $\mathcal{X}$
 - e.g., market conditions, space of capital, opponents' state, etc.
- The **economic-model error**: $\ell(x, f)$
 - e.g., Euler and Bellman residuals, equilibrium FOCs.

Then a **solution** is $f^* \in \mathcal{F}$ where $\ell(x, f^*) = \mathbf{0}$ for all $x \in \mathcal{X}$.

Approximate solution

1. Sample $\mathcal{X}$: $\mathcal{D} = \{x_1, \dots, x_N\}$
2. Pick a family of parametric functions (e.g., polynomials, Chebyshev polynomials, neural networks) $f(\cdot; \theta)$:
 - $\theta = \{\theta_1, \dots, \theta_M\}$: parameters for optimization (e.g., coefficients, or weights and biases).
3. To find an approximation for f solve:

$$\min_{\theta} \frac{1}{N} \sum_{x \in \mathcal{D}} \underbrace{\| \ell(x, f(\cdot; \theta)) \|_2^2}_{\text{Econ model error}}$$

- We call this objective the **loss function** $\mathcal{L}(\theta; \mathcal{D}) = \frac{1}{N} \sum_{x \in \mathcal{D}} \|\ell(x, f(\cdot; \theta))\|_2^2$.
- Evaluated on the training sample $\mathcal{D}$ it is the **train loss**; on test points not used in training $\mathcal{D}_{\text{test}}$ it is the **test loss**.

Neural Networks and Overparameterization

Neural Networks form a **highly-overparameterized** ($M \gg N$) class of functions.

- Example: one layer neural network, $f(\cdot; \theta) : \mathbb{R}^Q \rightarrow \mathbb{R}$:

$$f(x; \theta) = W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2$$

- $W_1 \in \mathbb{R}^{P \times Q}$, $b_1 \in \mathbb{R}^{P \times 1}$, $W_2 \in \mathbb{R}^{1 \times P}$, and $b_2 \in \mathbb{R}$.
- $\theta \equiv \{b_1, W_1, b_2, W_2\}$ are the coefficients, in this example $M = PQ + P + P + 1$.
- $\sigma(\cdot)$ is a nonlinear function applied element-wise (e.g., $\tanh(\cdot)$).
- Making it “deeper” by adding another “layer”: $f(x; \theta) \equiv W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2) + b_3$.

Deep learning is a **non-convex** optimization

- Training a neural network is **non-convex** in θ on its own (even with linear neural networks).
- On top of that, most economic dynamic models are **nonlinear**.
- In a non-convex problem, the solution depends on the **initialization** of the weights and biases θ .
- So the “answer” can depend on the initialization.
- We run the algorithm for **50** different initializations (seeds) of the weights and biases.

Applications

Experimental design

- We demonstrate these properties across four models of increasing difficulty:

1. Linear-Quadratic

Closed-form solution. Sharpest test of approximation error.

2. McCall Job Search

1D state, kink at reservation wage. Non-smooth approximation.

3. Real Business Cycle

2D stochastic state. No closed form.
Benchmarked vs. VFI.

4. Small Open Economy

5D state, three AR(1) shocks. Beyond any grid-based benchmark.

- Benchmarks where they exist (LQ, McCall, RBC); Euler residuals certify accuracy where none does (SOE).
- In all four: **accuracy** and **stability** improve with overparameterization.

Application 1: Linear-Quadratic regulator

The LQ problem

- The general deterministic LQ problem:

$$\begin{aligned} v(x) &= \max_u \{ -x^\top R x - u^\top Q u + \beta v(x') \} \\ \text{s.t. } x' &= Ax + Bu, \quad x_0 \text{ given} \end{aligned}$$

- State variable: $x \in \mathbb{R}^n$. Control variable: $u \in \mathbb{R}^m$.
- Closed-form solution:

$$v(x) = -x^\top P x, \quad u(x) = -F x$$

where P and F are obtained from the discrete-time algebraic Riccati equation.

- Sharpest test: approximation error can be computed exactly at every point.

A competitive firm with adjustment costs

- A price-taking firm in a competitive industry.
- Own output y , aggregate output Y (exogenous). Inverse demand: $\alpha_0 - \alpha_1 Y$.
- The firm chooses investment u subject to quadratic adjustment cost $\frac{\gamma}{2}u^2$:

$$\begin{aligned} v(y, Y) = \max_u & \left\{ (\alpha_0 - \alpha_1 Y)y - \frac{\gamma}{2}u^2 + \beta v(y', Y') \right\} \\ \text{s.t.} \quad & Y' = h_0 + h_1 Y, \quad y' = y + u \end{aligned}$$

- State is two-dimensional: (y, Y) . With augmented state $x = (1, y, Y)^\top$, this maps into the general LQ framework.
- Euler equation:

$$\gamma u(y, Y) = \beta [\gamma u(y', Y') + (\alpha_0 - \alpha_1 Y')]$$

LQ: neural network implementation

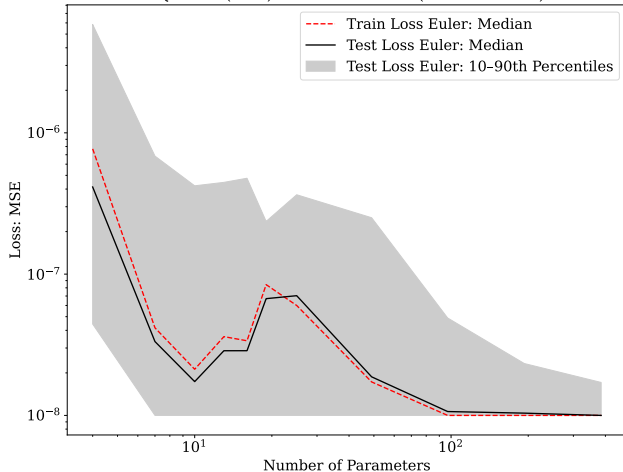
- **Architecture:** one-hidden-layer Tanh networks for value and policy functions. Policy network widths $H = 1$ to $H = 128$ (4 to 385 parameters).
- **Training data:** the optimal u depends only on Y (y enters linearly and does not affect the marginal return to investment), so we minimize the mean squared Euler residual over $n_Y = 6$ grid points:

$$\min_{\theta_u} \frac{1}{n_Y} \sum_{i=1}^{n_Y} [\gamma u(Y_i; \theta_u) - \beta(\gamma u(Y'_i; \theta_u) + \alpha_0 - \alpha_1 Y'_i)]^2$$

- Adam optimizer (up to 50,000 epochs) with early stopping at loss $< 10^{-8}$.
- **Test data:** $T = 29$ period path simulated from fixed initial conditions; apart from the initial point, the path lies strictly between training nodes.
- **Accuracy metric:** absolute relative error $\varepsilon_u(Y) = |u(Y; \theta_u^*) - u(Y)|/|u(Y)|$. Summarized by max and median across the test path, then 10th/50th/90th percentiles across 50 seeds.

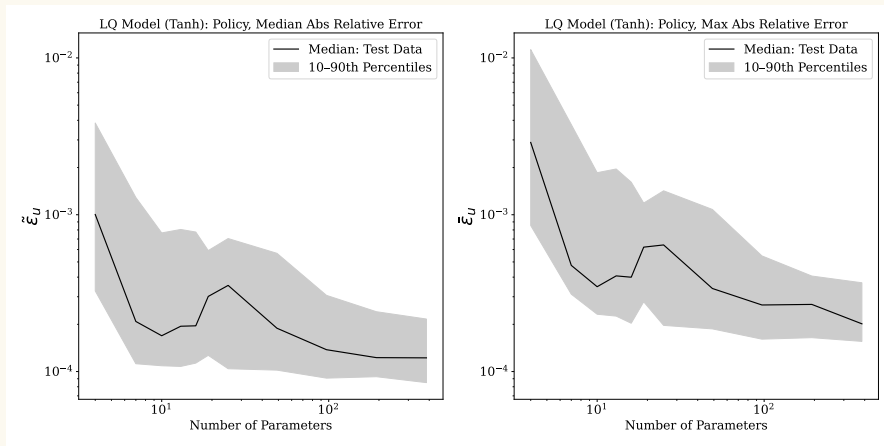
LQ: train and test Euler-residual loss

LQ Model (Tanh): Train vs Test Loss (Euler Residuals)



- Train and test curves are indistinguishable at every width, and both reach the early-stopping tolerance (10^{-8}) once the network is wide enough.
- The across-seed band collapses as width grows.

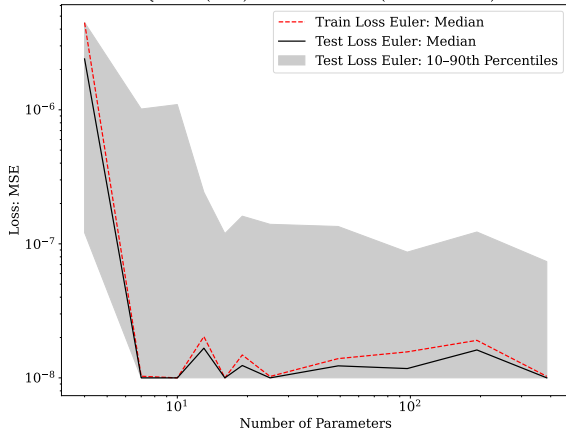
LQ: absolute relative error vs. closed-form solution



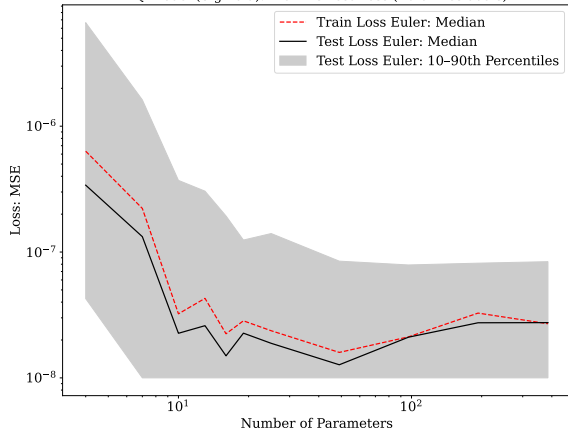
- Median (left) and max (right) relative error fall to $\sim 10^{-4}$ and stay there: the floor is set by the 10^{-8} residual tolerance, not by the network.
- The across-seed band narrows with width: accuracy **and** agreement improve together.

LQ: train and test loss, SiLU and Sigmoid

LQ Model (SiLU): Train vs Test Loss (Euler Residuals)



LQ Model (Sigmoid): Train vs Test Loss (Euler Residuals)



- The same pattern holds for other smooth activations: SiLU (left) and Sigmoid (right).

Application 2: McCall job-search model

From a linear to a non-smooth operator equation

- The LQ model is linear, which makes it the cleanest test but also the easiest.
- Does the blessing survive when the solution is nonlinear and non-smooth?
- The McCall model (McCall, 1970) provides exactly this test: the value function has a **kink** at the reservation wage, so the network must approximate a non-differentiable function.
- An unemployed worker receives i.i.d. wage offers $w \sim f$ on $[0, B]$ each period.
- Accept: permanent income stream $w/(1 - \beta)$. Reject: unemployment compensation c plus continuation value.
- Bellman equation (a nonlinear integral equation with a max operator):

$$v(w) = \max \left\{ \frac{w}{1 - \beta}, c + \beta \int_0^B v(w') f(w') dw' \right\}$$

McCall: the kink and why it matters

- Optimal policy: threshold form. There exists a reservation wage $\bar{w}$ such that accept iff $w > \bar{w}$.
- Closed-form value function:

$$v(w) = \begin{cases} \bar{w}/(1 - \beta) & \text{if } w \leq \bar{w}, \\ w/(1 - \beta) & \text{if } w > \bar{w}. \end{cases}$$

- The reservation wage $\bar{w}$ solves:

$$\bar{w} - c = \frac{\beta}{1 - \beta} \int_{\bar{w}}^B (w' - \bar{w}) f(w') dw'$$

LHS: flow gain from accepting the marginal offer. RHS: expected gain from continued search.

- The kink at $\bar{w}$ is a demanding test: the network must learn a non-smooth function from only $N = 11$ training points.

McCall: neural network implementation

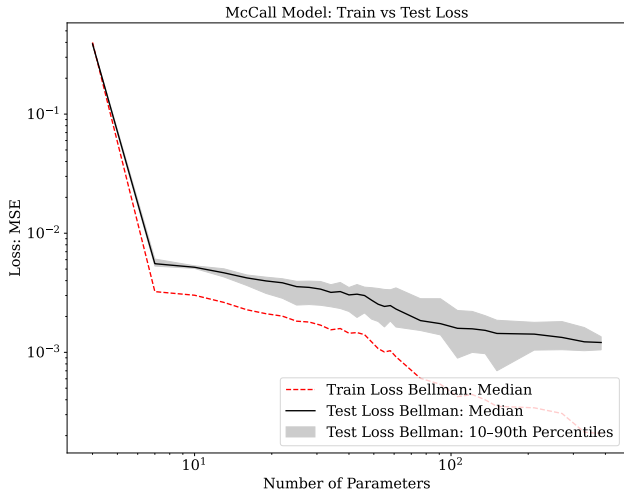
- **Architecture:** one-hidden-layer Tanh network $v(w; \theta_v)$, linear output. 50 random seeds per width.
- **Training:** $N = 11$ uniform grid on $[0, 1]$. Minimize mean squared Bellman residual:

$$\min_{\theta_v} \frac{1}{N} \sum_{w_i \in \mathcal{D}} \left[v(w_i; \theta_v) - \max \left\{ \frac{w_i}{1 - \beta}, c + \beta \int_0^B v(w'; \theta_v) f(w') dw' \right\} \right]^2$$

Integral by quadrature. Adam with early stopping at 10^{-8} .

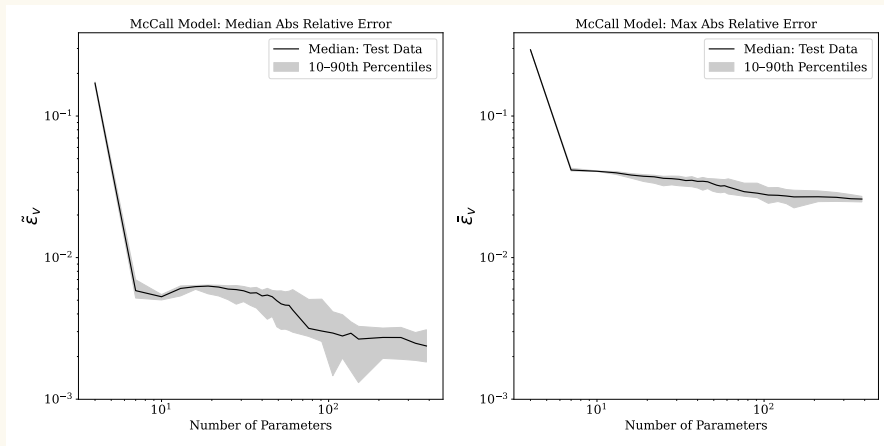
- **Test:** finer $N = 101$ uniform grid.
- **Accuracy metric:** absolute relative error $\varepsilon_v(w) = |v(w; \theta_v^*) - v(w)|/|v(w)|$. Summarized by max and median across test grid, then 10th/50th/90th percentiles across 50 seeds.

McCall: train and test Bellman-residual loss



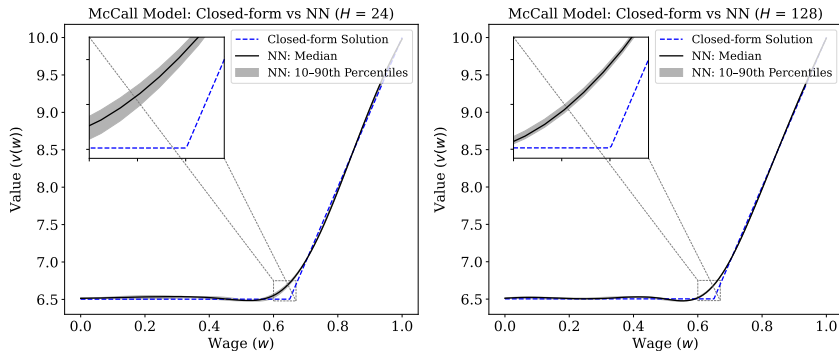
- Test loss falls monotonically through the overparameterized regime.
- The test plateau is the kink's floor, not the network's.
- The across-seed band collapses.

McCall: absolute relative error



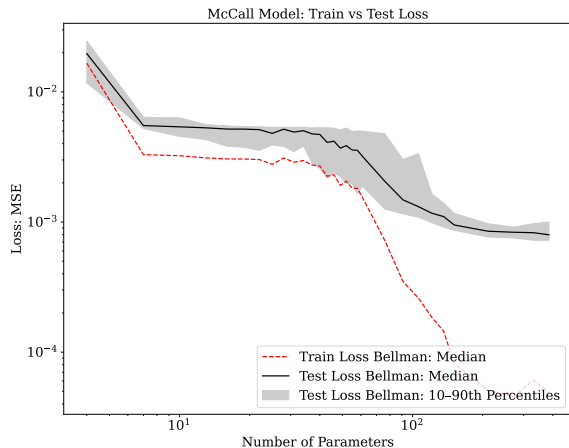
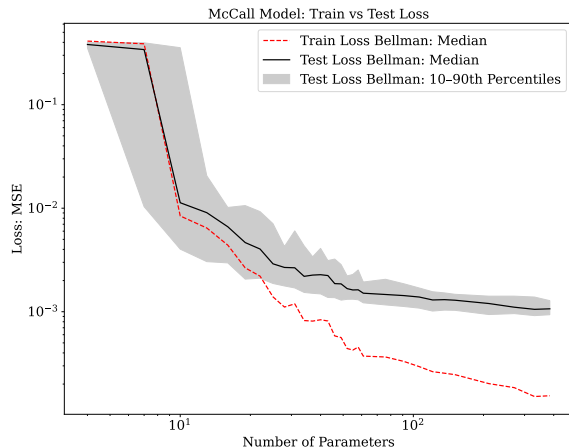
- Median relative error (left) falls from $\sim 17\%$ to $\sim 0.2\%$; max (right) from $\sim 29\%$ to $\sim 2.6\%$, its plateau set by the kink.
- The across-seed band narrows sharply: **stability blessing confirmed.**

McCall: value function, narrow vs. wide network



- Left: $H = 24$. The median tracks the closed-form solution, but the across-seed band is widest around the kink.
- Right: $H = 128$. The band collapses: 50 random initializations converge to the same function, with only the corner at $\bar{w}$ gently rounded.

McCall: train and test loss, SiLU and Sigmoid



- The same pattern holds for other smooth activations: SiLU (left) and Sigmoid (right).

Application 3: Real Business Cycle model

From deterministic to stochastic, from closed form to none

- The LQ model is linear with a closed-form solution. The McCall model is nonlinear with a kink but still one-dimensional and still admits a closed form.
- The Real Business Cycle model (Kydland and Prescott, 1982) is a demanding test: a two-dimensional stochastic state space (k_t, z_t) and no analytical solution.
- Social planner, Cobb–Douglas production $e^{z_t} k_t^\alpha$, stochastic TFP:

$$\begin{aligned} \max_{\{c_t, k_{t+1}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + k_{t+1} &= e^{z_t} k_t^\alpha + (1 - \delta)k_t \\ z_t &= \rho z_{t-1} + \sigma \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1) \end{aligned}$$

- Key object: policy function $k'(k, z)$. Benchmark: value function iteration (VFI) on a fine grid.

RBC: the Euler equation

- Euler equation:

$$u'(c_t) = \beta \mathbb{E}_t[u'(c_{t+1}) (\alpha e^{z_{t+1}} k_{t+1}^{\alpha-1} + 1 - \delta)]$$

- **LHS**: marginal utility cost of saving one unit today.
- **RHS**: expected discounted marginal utility gain from extra capital tomorrow (marginal product plus undepreciated capital).

RBC: neural network implementation

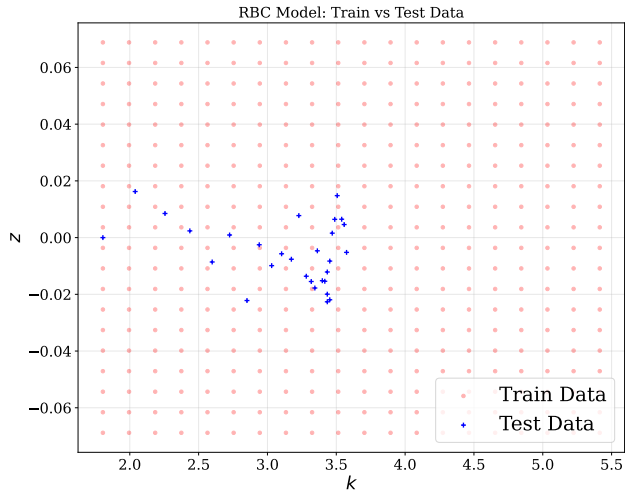
- **Architecture:** one-hidden-layer SiLU network $k'(k, z; \theta)$. Output layer: Softplus activation $\log(1 + e^x)$ to enforce $k' > 0$. 50 seeds per width.
- **Training:** 20×20 Cartesian grid ($N = 400$). Euler residual:

$$\ell_{\text{Euler}}(k, z; \theta) = \log \left(\beta c(k, z; \theta) \sum_{i=1}^J \frac{w_i}{\sqrt{\pi}} \left[\frac{\alpha e^{\rho z + \sqrt{2}\sigma\zeta_i} k'(k, z; \theta)^{\alpha-1} + 1 - \delta}{c(k'(k, z; \theta), \rho z + \sqrt{2}\sigma\zeta_i; \theta)} \right] \right)$$

with $J = 15$ Gauss–Hermite nodes. Adam with StepLR scheduler.

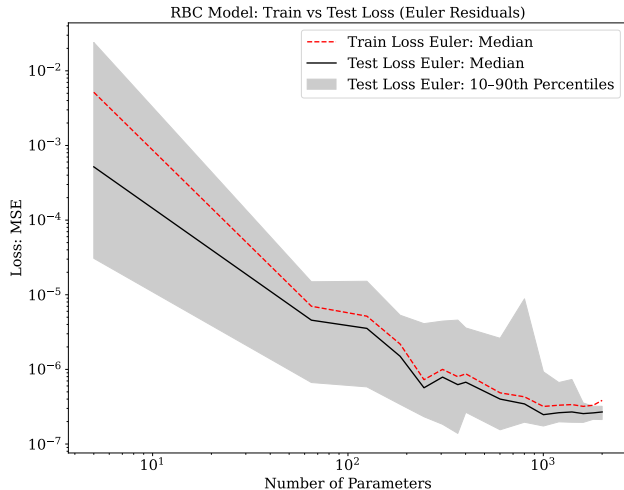
- **Penalty term:** $\lambda (k'(k^*, 0; \theta) - k^*)^2$ with $\lambda = 0.01$, enforcing a necessary implication of the transversality condition at the steady state.

RBC: training and test data



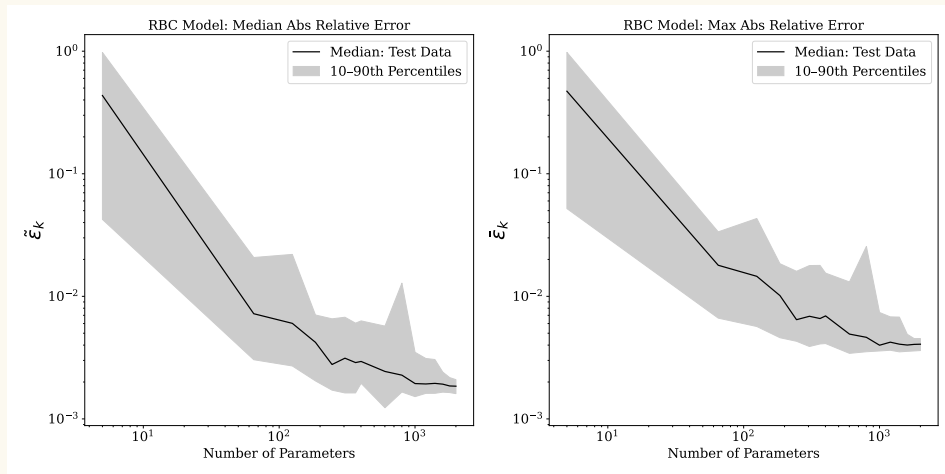
- Red dots: 20×20 training grid.
- Blue crosses: $T = 29$ path simulated under the VFI policy starting at $k_0 = 0.5 k^*$.
- Test path visits the interior of the state space and is never on a training node.

RBC: train and test Euler-residual loss



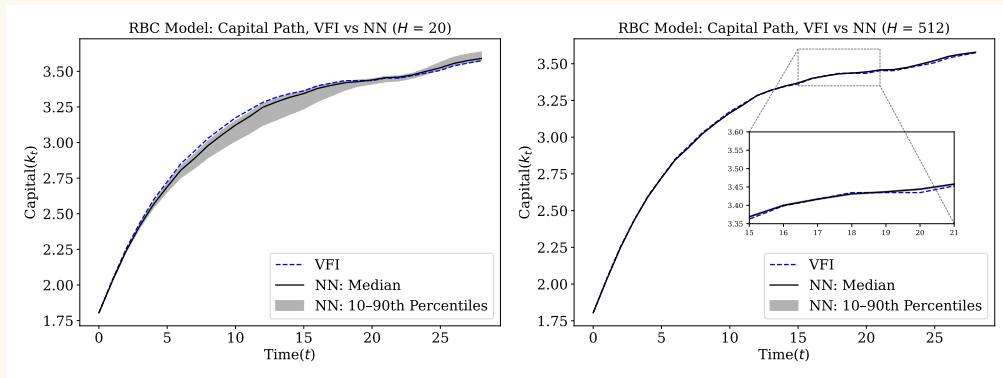
- Both losses decline as the network grows wider, with no **sign of overfitting**.
- The across-seed band narrows sharply: overparameterization simultaneously improves accuracy and reduces initialization sensitivity.

RBC: absolute relative error vs. VFI



- Median (left) and maximum (right) absolute relative error of the NN capital policy versus VFI.
- Both statistics decline and concentrate around the median seed as width increases.

RBC: capital paths, underparameterized vs. overparameterized



- Left: $H = 20$. Median path close to VFI, but across-seed band is wide. Accurate on average but unreliable.
- Right: $H = 512$. Median tracks VFI almost exactly over the full horizon and the band collapses to near zero.

Why it works: a random-features dissection

The lab: random features on the LQ model

- In the deep learning results, **something** selects one solution among many. To see what, we strip the model down to a case where nothing is hidden.
- Random features:** draw the first layer $(\tilde{w}_j, \tilde{b}_j)_{j=1}^H$ once, **freeze it**, and train only the readout $\theta = (v_1, \dots, v_H, c)$:

$$f_H(Y; \theta) = \sum_{j=1}^H v_j \sigma(\tilde{w}_j Y + \tilde{b}_j) + c$$

- The LQ Euler residual is **linear in** θ : stacking the N collocation points, solving the model reduces to a **linear system**

$$\text{LQ with random features} \implies \Phi \theta = d$$

where $\Phi \in \mathbb{R}^{N \times (H+1)}$ depends only on the data points and the frozen $\tilde{w}, \tilde{b}$.

- Same design as before: $N = 6$ collocation points, $H = 1$ to 1000 (50 seeds).

Infinitely many solutions: the algorithm chooses

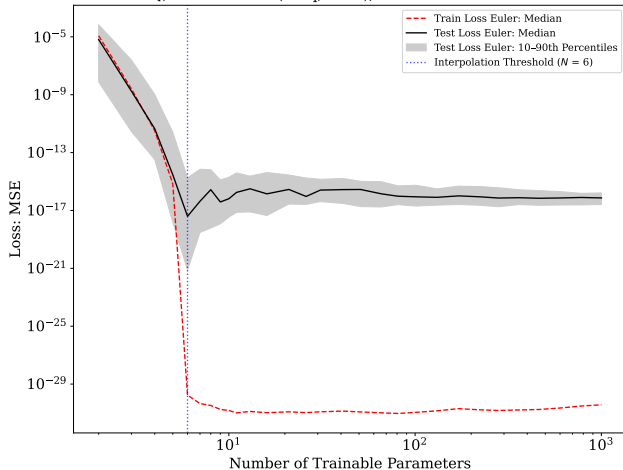
- For $H + 1 > N$, the system $\Phi\theta = d$ has **infinitely many exact solutions**.
- The loss cannot identify θ . **The algorithm is the identifying assumption.**
- **Selection theorem:** gradient descent from zero initialization converges to the minimum-norm interpolant

$$\theta^* = \Phi^\top (\Phi \Phi^\top)^{-1} d = \underset{\theta: \Phi\theta=d}{\operatorname{argmin}} \|\theta\|_2,$$

So instead of running gradient descent, we compute θ^* directly with a least-squares solver (`lstsq`) in one linear solve, **bypassing training entirely**. This removes arrival and trainability, leaving only the solution the implicit bias selects.

The blessing, in the lab

LQ, RF Min-Norm (lstsq, Tanh), $N = 6$: Train vs Test Loss

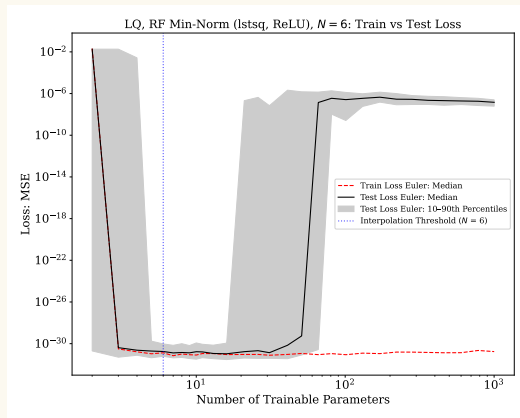


- Tanh features, minimum-norm solution: past the threshold the train loss is **exactly zero** (machine precision) at every width.
- Test loss stays flat to $H = 1000$: **test error is insensitive to width**.

Minimum norm knows no economic model

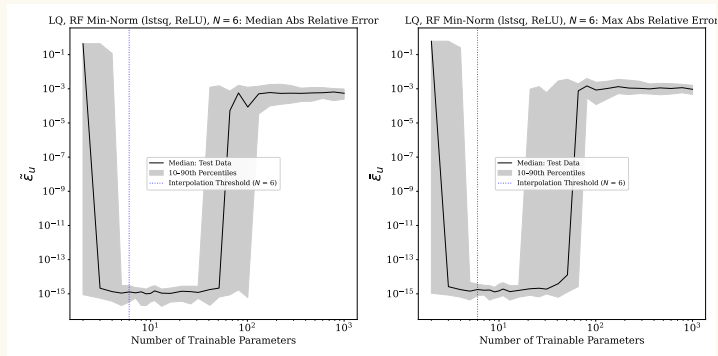
- The algorithm resolves the indeterminacy by minimizing $\|\theta\|_2$: a criterion in **parameter space**.
- It knows nothing about the economics: nothing about the Euler equation **between the grid points**, nothing about the smoothness of the true policy.
- The training loss is zero and the algorithm has picked one solution. **Is it the one we want?**

ReLU random features: train and test loss



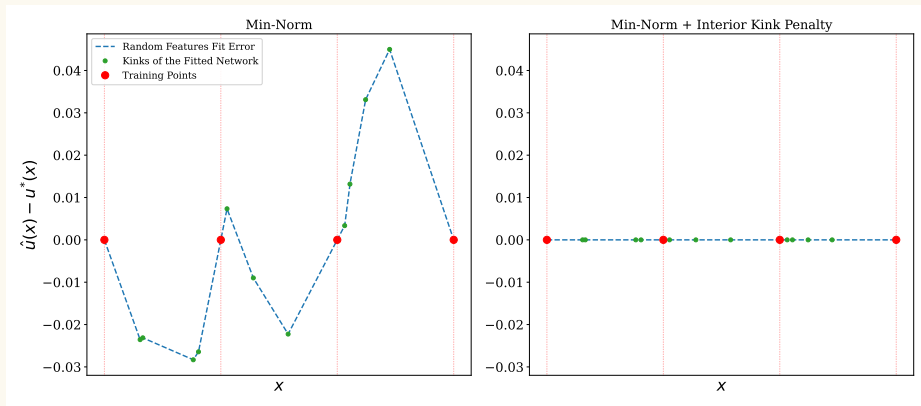
- Train loss is pinned at machine zero past the threshold, at every width; the test loss departs around $H \approx 65$ and plateaus: the bias is visible **out of sample** while the training loss stays silent.

ReLU random features: relative errors



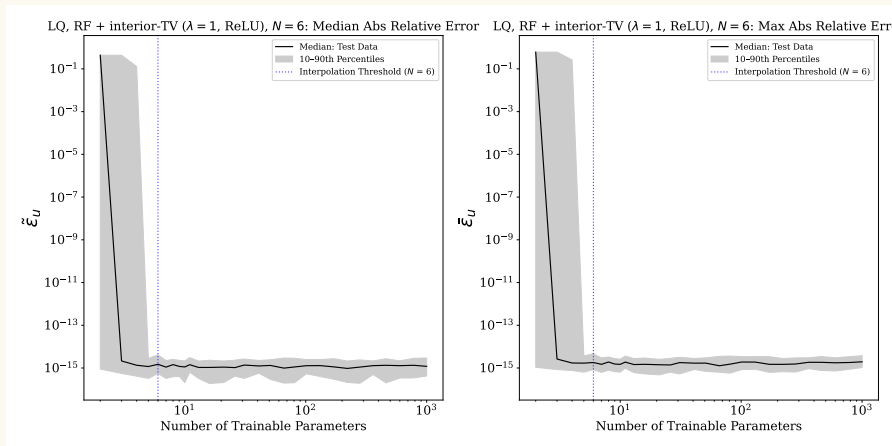
- Exact (10^{-15}) at small H , then the error **rises** with width and stays up.
- Minimum norm in **coefficients** is not smoothness in **functions**: kinked features inject curvature between grid points, but the truth is linear.

The mechanism, in one picture



- The min-norm fit is exact at the data points but **kinks between them**: that is where the error lives.
- The cure: **penalize the kinks between the points**. The fit snaps back to the line (10^{-15}).

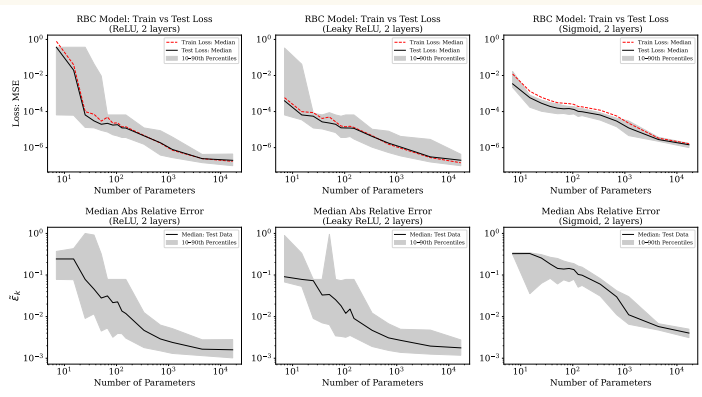
The cure, in the lab



- Linear quadratic with random features, with the **interior penalty**: flat at 10^{-15} at every width.

Robustness

Robustness: deeper networks for the RBC model



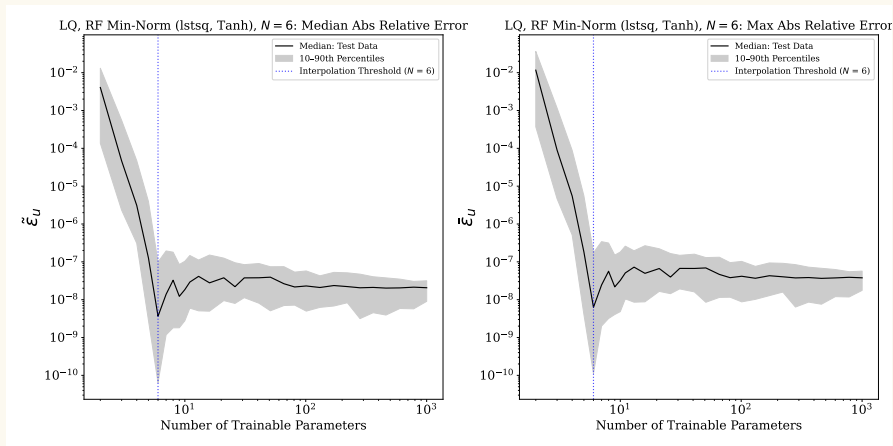
- Two-hidden-layer architecture with ReLU (left), Leaky ReLU (center), Sigmoid (right).
- All three activations replicate the main finding for a stochastic model with no closed-form solution.

Conclusion

- **Finding 1:** as network width increases, out-of-sample Euler and Bellman residuals decline. Value and policy functions converge toward their benchmarks.
- **Finding 2:** algorithmic stability. Small networks produce solutions that depend on the **initialization**. Wide networks concentrate across seeds, making the algorithm reliable and **initialization-independent**.
- **Finding 3:** a mechanism, exposed by a random-features dissection: **width buys agreement; the implicit bias decides what you agree on**.
- Documented consistently across four models: linear with closed form (LQ), nonlinear with kink (McCall), stochastic with no analytical solution (RBC), and high-dimensional (Small Open Economy); robust across smooth activations (Tanh, SiLU, Sigmoid) and depths.
- The moral: the parameter count was never the risk.

Appendix

RF lab (tanh): absolute relative error



- Relative errors tell the same story: flat from the threshold on, with a narrowing across-draw band.

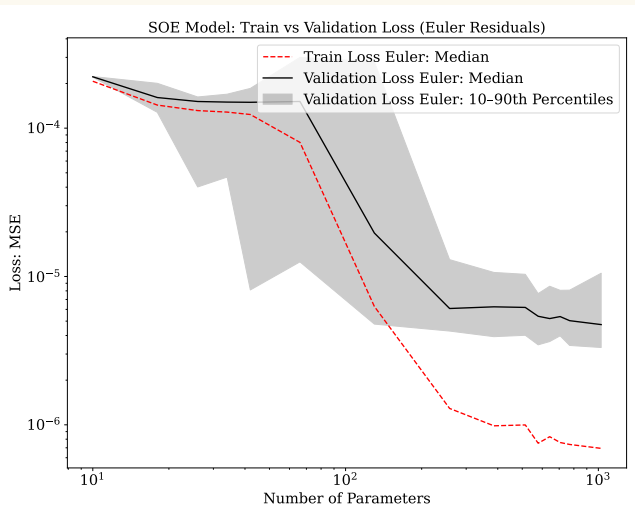
What the penalty is, exactly

- Each ReLU unit is a kink: at $Y_j = -\tilde{b}_j/\tilde{w}_j$ the slope of the fit jumps by $v_j\tilde{w}_j$.
- Charge for the slope jumps that land **inside the training range**:

$$\min_{\theta} \|\Phi\theta - d\|_2^2 + \lambda \sum_{j: Y_j \in (Y_{\min}, Y_{\max})} (v_j\tilde{w}_j)^2, \quad \lambda = 1$$

- In function space this is a **measure of curvature**: u'' of a ReLU fit is a point mass at each kink, of size the slope jump. The sum of jumps is $\int |u''|$ over the training range.
- Zero curvature inside the data means linear inside the data.

Small Open Economy: train and validation loss



- Small open economy ([Mendoza and Villalvazo, 2020](#)): five-dimensional state (capital, bonds, three AR(1) shocks).
- No grid-based benchmark exists at this dimension; the Euler residuals themselves certify accuracy.
- Train and validation loss decline together as the network grows: the blessing at five dimensions.

LQ: value function

- So far: the policy u . The same logic applies to the **value function** $v(y, Y)$, trained on the Bellman residual.
- **Two-stage pipeline:** first train a policy network $u(\cdot; \theta_u^*)$ ($H = 8$, fixed seed); then, holding it fixed, train $v(y, Y; \theta_v)$:

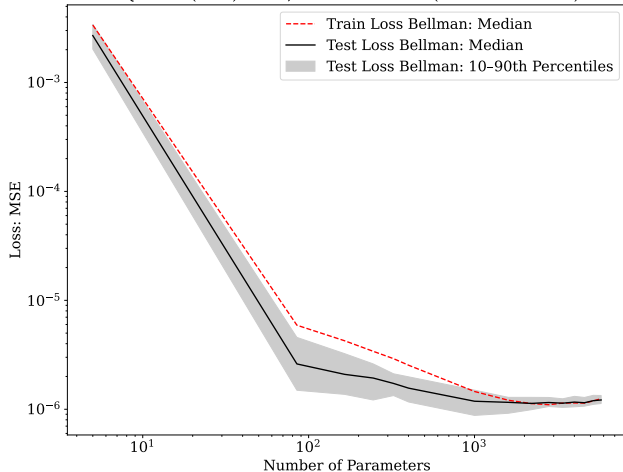
$$\min_{\theta_v} \frac{1}{N} \sum_{(y_i, Y_i) \in \mathcal{D}} \left[v(y_i, Y_i; \theta_v) - \left(-\frac{\gamma}{2} u(Y_i; \theta_u^*)^2 + (\alpha_0 - \alpha_1 Y_i) y_i + \beta v(y'_i, Y'_i; \theta_v) \right) \right]^2$$

where $y'_i = y_i + u(Y_i; \theta_u^*)$ and $Y'_i = h_0 + h_1 Y_i$; no gradient flows to the policy.

- **Training:** 12×12 Cartesian grid on (y, Y) , $N = 144$. Adam with early stopping at 10^{-8} . Widths H up to 1450 (5801 parameters).
- **Test:** $T = 20$ path $(\hat{y}_t, Y_t)$ simulated under the closed-form policy; benchmark is the closed-form value function.

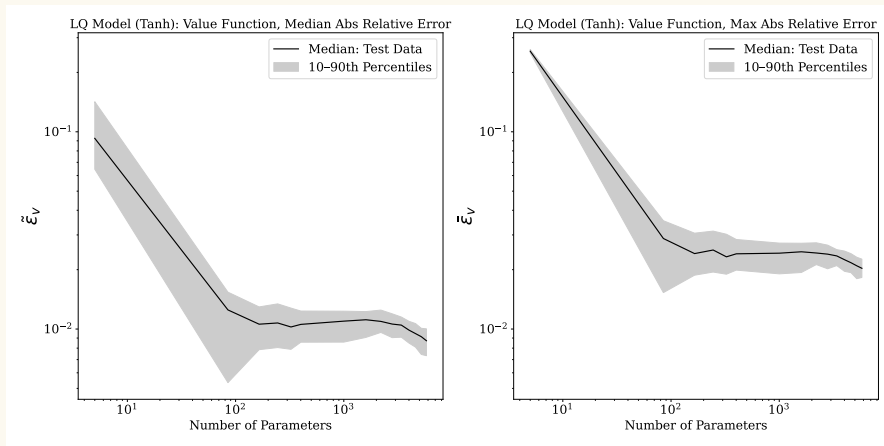
LQ: value function, train and test Bellman-residual loss

LQ Model (Tanh): Value, Train vs Test Loss (Bellman Residuals)



- Train and test loss fall together by three orders of magnitude and track each other at every width: no overfitting out to 5801 parameters ($N = 144$).
- The across-seed band narrows as the network grows.

LQ: value function, absolute relative error



- Median relative error (left) falls from $\sim 9\%$ to $\sim 1\%$; max (right) from $\sim 26\%$ to $\sim 2\%$, versus the closed-form value function.
- Accuracy and across-seed agreement improve with width.

Parameterization: Linear-Quadratic model

Description	Symbol	Value	
Discount factor	β	0.9	
Demand intercept	α_0	1.0	inverse demand level
Demand slope	α_1	1.3	price sensitivity to aggregate
Adjustment cost	γ	100	convex investment cost
Aggregate drift	h_0	0.05	constant in $Y' = h_0 + h_1 Y$
Aggregate persistence	h_1	0.9	AR coefficient for Y

Parameterization: McCall model

Description	Symbol	Value	
Discount factor	β	0.9	
Wage support	B	1.0	upper bound of $U[0, B]$
Unemployment benefit	c	0.1	flow payoff while searching

Parameterization: Real Business Cycle model

Description	Symbol	Value	
Discount factor	β	0.96	
Capital share	α	1/3	Cobb–Douglas exponent
Depreciation rate	δ	0.1	per-period capital depreciation
TFP persistence	ρ	0.9	AR(1) coefficient for z_t
TFP volatility	σ	0.01	std. dev. of ε_t
Utility function	$u(c)$	$\ln c$	log utility

Activation functions

For a scalar pre-activation $z \in \mathbb{R}$, applied element-wise:

- **ReLU:**

$$\sigma(z) = \max\{0, z\}$$

- **Leaky ReLU** (slope $\alpha = 0.01$):

$$\sigma(z) = \max\{\alpha z, z\}$$

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **SiLU:**

$$\sigma(z) = \frac{z}{1 + e^{-z}}$$

- **Tanh:**

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$